

Things to Know for this Lab

All writing and reading requires including `stdio.h`:

```
#include<stdio.h> /* this is a comment line (can be on separate line too) */
```

Basic structure of a C program (call it `foo.c`):

```
#include....
int main(){ /* start of the main function. All execution starts here */
    .... /* code */
    return (...) /* return value to the operating system */
} /* end of the main program */
```

To compile the code: `gcc foo.c -o foo` compiles the program `foo.c` and creates the executable `foo`.

Most programs we write will also require `math.h`, which is the include file for the `libmath.so` library. The 'so' indicates that the library is a "shared" library, which means that a single copy of the library function is in the memory which is used by all programs that require it.

To compile such a program, we have to teach the compiler that we need `libmath.so`. For that we execute: `gcc foo.c -lm -o foo`. (Actually the current gcc on my computer seems to use `libmath.so` automatically. So the `-lm` may not be required.)

Printf:

To write something, use `printf("...")`.

```
printf("This is a line\n");
This is a line
```

To print multi-line output

```
printf("This is\na line\n"); /* each \n starts a new line */
This is
a line
```

Note: if you had written

```
printf("This is\n a line\n");
```

A space would have been added ahead of the 'a' in the second line.

Integer variables and assignment and printing the same:

To declare an integer variable and assign it a value

```
int a;
a=4;
```

Can also assign the value in the same line:

```
int a=4;
```

To write something that includes an integer variable (printf conversion specification for integer is %d)

```
printf("The value of a = %d\n",a); /* output printed, with %d replaced by value */
The value of a = 4
```

Note that the following is allowed: %6d - print as integer, using 6 spaces

```
int a=8, b=120;
printf("The value of a = %3d and b = %3d\n",8,15); /* use 2 digits for all output */
The value of a = 8 and b = 12                      /* note extra space for 8 */
```

Addition of variables and constants

```
Int b=10;
Int c=15; /* can give the variable a value inside the declaration itself */
Int d,e;
d=a+b+c; /* addition of variables */
e=a+4+c; /* addition of variables and constants (here 4 is a constant) */
```

String constants, declaration and output.

To declare a string variable, and assign it a value

```
char name[ ] = "Abc Xyz";
```

The syntax will be explained later, but this is a simple way to declare string constants. To print some strings, use

```
printf("My name is %s, and my home is in %s\n", name, city);
```

%s is the symbol used to indicate a string. It will be taken from the list of variables following the format string. Again, we can modify the output of the string:

```
printf("My name is %10s, and my home is in %10s\n", name, city);
```

This limits the number of spaces to be used for name and city to be a maximum of 10. If the string has 10 spaces or less, it will print out correctly with additional spaces as required. But if the string is larger, C will print out the whole string so that the output is readable.

Float and Double variables, assignment and output:

To declare a real number, and assign it a value

```
float a = 4.5;
```

This creates a real number that uses 32 bits of storage. The mantissa is 24 bits in size including sign, and the exponent is 8 bits in size. Note that exponent cannot exceed 7 bits as there is an offset. To print float numbers use

```
printf("I am %f cm tall\n", 66.5);
```

Note that the value for height can also be taken from a constant

Long long int variables:

To declare a long int variable. This is a little strange. "int" is not a fixed size. It depends on the cpu of the computer. So a long integer variable is called "long long int"

```
long long int aa = 123456789;  
printf("aa = %lld\n", aa);
```

This creates an integer that uses 64 bits of storage. Such an integer can store much larger size integer values. For example,

```
long long int amnt=1000; /* amount in crores to be printed out */  
int crore = 10000000; /* Number of Rs in 1 crore */  
amnt = amnt*crore; /* amnt converted to Rs. */  
printf("1000 crores is %lld Rs.", amnt );
```

To declare a long real number, and assign it a value

```
double b = 3.5e250
```

This creates large real numbers whose mantissa or exponent will not fit into a float variable.

Arrays

Arrays are a sequence of numbers or strings. For example:

```
int a=[1,2,3,4,5]; /* array allocated and initialized */  
int b[10]; /* array allocated but values not given */
```

The type of variable is not constrained. It can be anything. For example we can have strings, which are arrays of characters. We can also have arrays of strings, which means it is an array of arrays of char. Etc.

Arrays are very frequently used together with for loops.

Scanf:

If we want the user to enter some information, we use the scanf function. The format is

```
scanf("format",&arg1, &arg2, ..., &argn);
```

For example, (assuming #include<stdio.h> is present at the top)

```
int a,b;
printf("enter a and b: "); /* help message */
scanf("%d %d", &a, &b);
printf("a = %d, b = %d \n",a,b);
```

The '&' symbol is needed to give scanf not the variable, but the memory location where the variable exists. Printf takes the value present at a memory. So the variable name is all that is needed. But here, we need the memory location so that scanf can write the value entered by the user into that memory.

gets and fgets:

Scanf is fine for entering numbers. But when it comes to strings, it is more difficult. Better to use gets or fgets. These are functions that read in an entire string. Usage is as follows:

```
char *gets(char *s); /* very unsafe and should not be used, as it assumes s has
enough space to accept whatever string entered by the user. DO NOT USE */
char *fgets(char *s, int size, FILE *stream);
/* size is the size of s and fgets will only read upto size-1 characters and add \0
At end. Returns the string if success and NULL if error. */
```

Example: Read a name from stdin:

```
char name[10];
printf("Enter your name: ");
fgets(name,10,stdin);
```

```
printf("Name entered: %s\n",name);
```

The remainder function:

We have already looked at +, -, * and /. An additional operator is the modulus operator. $x\%y$ is defined to be the remainder when x is divided by y . Both / and % are machine dependent when the arguments are negative.

Logical operators:

Relational operators are operators that act on logical expressions to get logical answers. If $i=4$ and $j=7$, then

$i < j$ is True

$2*i < j$ is false.

$(i < j) \&\& (2*i > j)$ is True as both statements are true

These will be very important once we talk about control statements.

Control Statements: if, for and while and switch:

```
if( logical expression ){  
    Code to be executed if TRUE  
}
```

```
if( logical expression ){  
    Code to be executed if TRUE  
}else{  
    Code to be executed if FALSE  
}
```

```
if( logical expression 1 ){  
    Code to be executed if logical expression 1 is TRUE  
}else if( logical expression 2 ){  
    Code to be executed if logical expression 2 is TRUE  
}else{  
    Code to be executed if logical expression 1 is FALSE  
}
```

IF statements can be nested. Eg from previous assignment:

```
if( D >= 0.0 ){  
    if( D == 0.0 ){/* nested if */  
        printf("The roots are identical \n");  
        ... code to find the roots' values
```

```

        } else {          /* this else goes with nested if */
            printf("The roots are real and distinct\n");
            ... code to find the roots' values
        }
    } else {              /* this goes with outer if statement */
        printf("The roots are complex and distinct\n");
        ... code to find the roots
    }
}

```

While and for:

```

while( logical expression ){ /* starts loop */
    Code                      /* must modify logical expression to terminate the loop */
}

```

```

for( expr 1 ; logical expr 2 ; expr 3 ){
    Code                      /* executed till expr2 becomes false */
}

```

For is the same as the following code:

```

Expr 1;
while( logical expr 2 ){
    Code;
    Expr 3;
}

```

Example: print all the digits in order:

```

printf("digits: \n");
for( int i=0 ; i<10 ; i++ ){
    printf("%1d\n",i);
}

```

New additions to this section:

More Control structures:**Conditional Expression: (p 51-52 of K&R)**

`(expr1)? Expr 2 : expr 3;`

Eg. `max(a,b)` is implemented in C as

`(a>b) ? a : b;`

Eg. printing 10 lines of an array per page:

`for( i=0 ; i<n ; i++ )`

`printf(“%6d%c”, a[i], (i%10==9 || i==n-1 ) ? \n : “ ”);`

The `%c` prints blanks except when `i%10` is 9 or `i` is `n-1`. In those cases inserts a `\n` (newpage)

Do While:

`do{`

Code

`}while( logical expression )`

It is like `while( expr ){`

Code

`}`

except that the Code is executed at least once.

Eg. To extract the digits of a positive number `s`: p64 of K&R

`Do{`

`S[i++] = n%10 + '0' /* get next remainder and convert into ascii */`

`}while( (n /= 10) > 0 ); /* divide n by 10, and loop if >0 */`

If `s` is less than 10, the division yields zero and terminates too soon. So the `do ... while`.

Break and Continue

If we want to exit the loop you are in prior to the exit condition, use BREAK. To merely skip the rest of the body of the loop and go to the next iteration, use CONTINUE.

Eg: remove trailing blanks, tabs and newlines. Line is in 's'.

```
for( n = strlen(s)-1 ; n>=0 ; n-- ){          /* count down n from length of string */
    if( s[n] != ' ' && s[n] != '\t' && s[n] != '\n' )
        break;                               /* exit the for loop if we have only blanks, tabs or \n */
}                                              /* indicates the loop that is skipped.*/
```

Eg: skip processing certain elements in a loop (p. 65 of K&R)

```
for( i=0 ; i<n ; i++ ){                    /* loop to process elements of an array */
    if( a[i]<0 )
        continue;                          /* do not process this element as it is -ve */
    ....                                    /* process remaining elements */
}
```

EVEN MORE Control Commands

Bitwise Operators:

& bitwise AND	$x = x \& \sim 077$ (zeros last 6 bits of x)
! bitwise (inclusive) OR	$x = x \! \text{ mask}$ (sets ON the 1 bits of the mask)
^ bitwise (exclusive) OR	$x \wedge y$ (sets to 1 all positions where x and y differ and 0 where they agree)

<< left shift x << 2 shifts bits in x to left, padding with zeros

>> right shift $x \gg 2$ shifts bits in x to right, padding with 0s.

~ one's complement of the argument ~x converts all 1 bits to 0 and 0 bits to 1

These operators have higher precedence than `&&`, `||`, `?:`, `=` etc but lower precedence than `==`, `!=`, `<`, `<=`, etc

These bits operations are useful to take apart the bits of a word. For instance if we know that the 4th bit of a variable x is 1 if a certain action is to be taken (very common in operating system situations), we “test” the variable by

```
if( x & 8 ){
```

...

}

Where 8 is 1000 in binary. `x&8` returns 1 if the 4th bit of `x` is 1, and zero otherwise. You will use this in the first question.

FUNCTIONS

A function is defined as

Type FunctionName(type arg1, type arg2, ...){

Definition of the function

...

...

}

The function must be declared prior to invoking it

A function can have local variables, and can reference any external (or global) variables (variables that have been defined outside any function, including main). Note that only external variables defined in the same file as the function can be accessed. If the main function is in another file, the external variables declared in that file are NOT accessible. (see p73 of the textbook)

Functions can call other functions, and can call themselves.

The arguments of a function are “passed by value”, which means that a copy of the variable is sent to the function. This means that the argument can be modified inside the function and thrown away. The results of the function can either be returned via the return (expression) command, or by storing the answer in an object whose address has been passed to the function.

In the problems here, both the strings and arrays are passed by address. So data can be returned through them. Whereas in the third problem, the status of the answer is returned by the return command.

Pointers

Int a,b; are variables that are stored in some memory locations

Int *p; is a pointer. Right now it has no value which means it might point to any address.

If I now type:

```
a = 4;
b = 6;
p = &a;          /* This means that pointer p now holds the address */
                  /* of variable a */
b += *p          /* This is the same as b += a, since p points to variable a */
```

So, in an expression, *p refers to the value in the memory address pointed to by p.

When defining the pointer,

```
Int *p;          /* means p is a pointer of type 'int'. */
```

Type of a pointer

The type of a pointer means the type of variable that is there at that address. A pointer could be pointing to a string:

```
char *p;
```

Or could be a pointer to double:

```
double *p;
```

Or it could be a pointer to an array of char (ie a string):

```
char *p="This is a string";
```

Or it could be a pointer to an array of a numeric type (say float):

```
int main(){
    float a[3],*q;
    a[0]=1.5;a[1]=2.6;a[2]=4.6;
    q=a;
    printf("q=%f, q+1=%f, q+2=%f\n",*q,*(q+1),*(q+2));
}
```

Output: q=1.500000, q+1=2.600000, q+2=4.600000

Pointer arithmetic:

If p points to a , then $p+1$ points to the next memory location to ' a ', of the "type" declared for ' a '. So since ' a ' has been declared float, $q+1$ will point to an address that is one float beyond the address of $a[0]$, i.e. it points to $a[1]$. Etc. So $p+i$ points to $a[i]$.

This makes pointers very powerful. All of array manipulation can now be done using pointer arithmetic. **Beware:** I can say

```
int main(){
    float a[3],*q;
    a[0]=1.5;a[1]=2.6;a[2]=4.6;
    q=a;
    printf("q=%f, q+1=%f, q+2=%f, q+3=%f\n",*q,*(q+1),*(q+2),*(q+3));
}
```

Here is what happens when I compile and run:

```
$ gcc ptr1.c          /* compiles without error */
$ ./a.out
q=1.500000, q+1=2.600000, q+2=4.600000, q+3=-36141.00 /* gives value for q+3! */
$ ./a.out            /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000,
q+3=192551726327398727155712.000000                /* different answer! */
$ ./a.out            /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000,
q+3=-11804037870771613760010517092440735744.000000 /* different answer! */
$ ./a.out            /* run again */
q=1.500000, q+1=2.600000, q+2=4.600000, q+3=-0.054069 /* different answer! */
```

Summary:

```
int *ip;          /* ip is a pointer to int */
ip = &x;          /* ip now points to x (say it holds 1) */
y = *ip;          /* y is now 1 */
*ip = 0;           /* x is now set to 0 */
ip = &z[0];        /* ip now points to z[0] */

y = *ip + 1;       /* *ip has highest precedence, so y = (*ip) + 1 */
*ip += 1;          /* increments z[0] by 1 */
```

However note that sometimes () are needed:

```
++*ip;            /* also increments z[0] */
(*ip)++;          /* also increments z[0] */
*ip++;            /* equivalent to *(ip++) since ++ has the higher precedence */
```

Problem 1: Write a C program called `ROLLNO_pointer_strlen.c` that inputs a string and prints out the length of the string. Take the following program on page 39 of the textbook and convert it to a version that uses only pointers and does not use any array indices.

```
/* strlen: return length of s (page 39 of K&R) */
Int strlen1(char s[]){
    Int i;
    i=0;
    while( s[i] != '\0' )
        ++i;
    return i;
}
```

Your program should receive a pointer, should not have array index 'i', and should not use index arithmetic. Instead it should use *s and can use a local pointer, and should only do pointer arithmetic.

HINT: pointers pointing to the beginning and end of a string are separated by the string's length, and if p and q are two pointers, p-q is the difference between the positions they point.

Example: string input is "Here is my version" the program should return
Length of "Here is my version" is 18 characters (apart from terminating '\0')

Problem 2. Write a C program called `ROLLNO_getline.c` that reads an input string from a file that has been opened as `FILE *iop`. It should be declared as

```
Int getline(char *line, int max, FILE *iop)
```

The function must use `fgets(line, max, iop)` for input and must store at most max-1 characters in the buffer "line", and returns the length of the line read in.

A version of `getline`, `fgets` and `fputs` are on page 165 of K&R. The `getline` there reads from `stdin`, whereas here you need to read from `FILE *iop`.

The code has to be changed so that any `\n` in the string is removed before being returned.

Note: Use the `ROLLNO_stdlen.c` (from Problem 1) to get the length of the string in this problem.

Example: string input is "Here is my version\n" and array is `char array[15]`, then you should return:
line contains "Here is my ver"

Problem 3: Change the `strcat` program below:

```
Void strcat(char s[], char t[]){
    Int i,j;
    I = j = 0;
    while( s[i] != '\0' )    /* find end of s */
        I++;
    while( (s[i++] = t[j++]) != '\0' ) /* copy t to end of s */
        ;
}
```

Into a pointer version of the same algorithm. Note that i and j should not be used, and neither should `s[i]` or `s[i++]` or `t[j++]`. Only pointers should be used.